Strings e f-strings

upper, lower, strip, split, replace, find, len e fatiamento

Aula 13 • 22/10/2026 • Linguagem de Programação Python (80h) • CETAM
Profa. Priscila Gonçalves

Visão geral da aula

Data

Quinta-feira, 22/10/2026
Aula 13 • 4 tempos
Laboratório de informática

Habilidade técnica

Aplicar os conceitos de estrutura de dados, organização e acesso de dados, com domínio de conhecimento.

Projeto

Padronizar o cadastro de clientes e montar o recibo da viagem

Habilidade técnica

- Aplicar os conceitos de estrutura de dados com domínio
- Utilizar funções de forma eficaz para manipular caracteres e palavras
- Padronizar e validar dados de texto
- Montar mensagens com f-strings

Objetos de conhecimento

- Estruturas de dados em Python: listas, tuplas e dicionários
- Métodos e funções
- Funções de manipulação de strings
- f-strings

Roteiro dos 4 tempos

1º tempo

Aula expositiva: funções para manipulação de strings

2º tempo

upper, lower, strip, split, replace, find, len, fatiamento e f-strings

3º tempo

Nomes, e-mail, CPF fictício e código de reserva

4º tempo

Cadastro padronizado, recibo com f-strings e revisão para a AV3

1

1º tempo • Funções de strings

Manipular caracteres e palavras

Retomada da Aula 12

- Aula 11: listas e métodos
- Aula 12: tuplas e dicionários
- Estruturas guardam e organizam dados
- Hoje: como tratar o texto dentro desses dados
- Nomes, e-mails, códigos e recibos são strings

O que é uma string?

- Sequência de caracteres entre aspas
- Cada caractere tem um índice, como na lista
- Acesso por índice e fatiamento
- len() informa o número de caracteres
- Pode ser percorrida com for

Índices e len em strings

```
nome = "Maria Silva"
print(nome[0])     # M
print(nome[-1])    # a
print(len(nome))   # 11
```

- Índice começa em 0
- Espaço também conta
- -1 é o último caractere

Strings são imutáveis

- Strings não mudam depois de criadas
- Métodos devolvem uma nova string
- Para guardar, atribua o resultado a uma variável
- nome[0] = "X" gera TypeError
- Mesma ideia das tuplas

Atribuir o resultado

```
nome = "maria"
nome.upper()
print(nome)   # maria
nome = nome.upper()
print(nome)   # MARIA
```

- Sem atribuir, nada muda
- O método devolve uma cópia
- Guarde na variável

Funções e métodos de string

- Função recebe a string: len(texto)
- Método usa ponto: texto.upper()
- Quase todos os métodos devolvem uma nova string
- Podem ser encadeados: texto.strip().title()
- Para explorar: help(str) no console

Quando manipular strings?

- Padronizar nomes e e-mails no cadastro
- Validar dados digitados pelo usuário
- Gerar códigos de reserva e identificadores
- Montar mensagens e recibos
- Limpar e separar textos

Aspas e caracteres especiais

- Aspas simples ou duplas formam uma string
- Aspas dentro do texto: use o outro tipo de aspas
- \n quebra a linha dentro do texto
- \t insere uma tabulação
- A barra invertida (\) é o caractere de escape

2

2º tempo • Demonstração

upper, lower, strip, split, replace, find, len, fatiamento e f-strings

upper, lower e title

```
nome = "Ana Paula"
print(nome.upper())   # ANA PAULA
print(nome.lower())   # ana paula
print(nome.title())   # Ana Paula
```

- upper: maiúsculas
- lower: minúsculas
- title: inicial de cada palavra

strip: remover espaços

```
nome = "   Ana Paula  "
print(nome.strip())
print(nome.lstrip())
print(len(nome))   # 14
```

- strip tira espaços das pontas
- lstrip e rstrip: um só lado
- Útil com o que o usuário digita

split: separar palavras

```
frase = "Ana Paula Souza"
partes = frase.split()
print(partes)
print(partes[0])    # Ana
print(len(partes))  # 3
```

- split devolve uma lista
- Sem argumento: separa por espaços
- split(",") separa por vírgula

replace: trocar trechos

```
tel = "92-99999-0000"
print(tel.replace("-", ""))
print(tel.replace("-", " "))
```

- Saída 1: 92999990000
- Saída 2: 92 99999 0000
- O original não muda

find: localizar um trecho

```
email = "ana@mail.com"
print(email.find("@"))   # 3
print(email.find("x"))   # -1
print("@" in email)      # True
```

- find devolve a posição
- -1 quando não encontra
- in só responde True ou False

len: tamanho do texto

```
senha = "python123"
print(len(senha))        # 9
print(len(senha) >= 8)   # True
```

- len conta os caracteres
- Base para validar tamanho
- Resultado do >= é bool

Fatiamento de strings

```
codigo = "MANAUS2026"
print(codigo[0:3])   # MAN
print(codigo[:6])    # MANAUS
print(codigo[-4:])   # 2026
```

- Mesmas regras da lista
- O fim não é incluído
- Negativo conta do fim

f-strings

- f-string: texto com valores dentro de chaves
- Escreve-se f antes das aspas: f"..."
- Aceita variáveis e expressões
- Mais legível que concatenar com +
- Permite formatar números, como {valor:.2f}

Primeiras f-strings

```
nome = "Ana"
dias = 4
preco = 600.0
print(f"{nome} viaja {dias} dias")
print(f"Total: R$ {dias * preco}")
```

- f antes das aspas
- Valores entre chaves
- Aceita expressões

Formatando números

```
valor = 1234.5
print(f"R$ {valor:.2f}")   # R$ 1234.50
print(f"{valor:10.2f}")    #    1234.50
```

- .2f: duas casas decimais
- Número antes: largura mínima
- Ótimo para valores em R$

Encadeando métodos

```
nome = "  maria silva "
limpo = nome.strip().title()
print(limpo)   # Maria Silva
print(len(limpo))   # 11
```

- strip e depois title
- Cada método age sobre o resultado
- Leia da esquerda para a direita

Erros comuns com strings

- Escrever Upper() em vez de upper()
- Esquecer os parênteses do método
- Esperar que o método altere a string original
- Esquecer o f antes das aspas na f-string
- find devolve -1 quando não encontra

3

3º tempo • Atividade prática

Nomes, e-mail, CPF fictício e código de reserva

Atividade prática

- Quatro exercícios com strings
- 1. Padronizar nomes
- 2. Validar e-mail
- 3. Validar CPF fictício
- 4. Gerar código de reserva

Exercício 1: padronizar nomes

```
nome = input("Nome: ")
nome = nome.strip().title()
print("Cadastro:", nome)
```

- Remove espaços e padroniza
- Teste: " jOÃO silva "
- Resultado: João Silva

Exercício 2: validar e-mail

```
email = input("E-mail: ").strip().lower()
if "@" in email and "." in email:
    print("E-mail válido")
else:
    print("E-mail inválido")
```

- Validação simples, didática
- Não substitui validação real
- Teste sem @ e sem ponto

Exercício 3: CPF fictício

```
cpf = input("CPF (só números): ")
if len(cpf) == 11 and cpf.isdigit():
    print("Formato válido")
else:
    print("CPF inválido")
```

- Use apenas dados fictícios
- isdigit: só dígitos
- len == 11 confere o tamanho

Formatando o CPF fictício

```
cpf = "12345678901"
fmt = f"{cpf[:3]}.{cpf[3:6]}.{cpf[6:9]}-{cpf[9:]}"
print(fmt)   # 123.456.789-01
```

Exercício 4: código de reserva

```
nome = "Ana Paula"
destino = "Tefé"
dia = 21
cod = f"{destino[:3]}{nome[:2]}{dia}"
print(cod.upper())   # TEFAN21
```

- f-string monta o código
- Fatias pegam as iniciais
- upper padroniza

Critérios da atividade

- Métodos de string usados corretamente
- Validações testadas com casos válidos e inválidos
- Código de reserva gerado de forma consistente
- Saída clara e conferida
- Comentários e nomes de variáveis claros

4

4º tempo • Desafio e AV3

Cadastro padronizado, recibo e revisão

Desafio: cadastro e recibo

- Padronizar o cadastro de clientes
- Nome com title() e e-mail em minúsculas, sem espaços
- Montar o recibo da viagem com f-strings
- Mostrar dias, gasto por dia e total com duas casas decimais

Exemplo de solução: recibo

```
nome = input("Nome: ").strip().title()
dias = int(input("Dias: "))
gasto = float(input("Gasto/dia R$: "))
total = dias * gasto
print("=== RECIBO ===")
print(f"Cliente: {nome}")
print(f"Dias: {dias}")
print(f"Total: R$ {total:.2f}")
```

- Padronização na entrada
- f-strings no recibo
- {total:.2f} formata o valor

Fechamento e revisão para a AV3

- Laços: while, for, break e continue
- Listas, tuplas e dicionários
- Strings e f-strings
- Dica: refaça os desafios sem consultar as soluções
- Tire as dúvidas antes da avaliação

Fechamento

- Hoje: strings, métodos e f-strings
- Prática: nomes, e-mail, CPF fictício e reserva
- Desafio: cadastro padronizado e recibo da viagem
- Boa revisão para a AV3.
- Obrigada!